

Aprendizado de Máquina B MLP

Lourenço R. G. Araújo - DEST
Pedro O.S. Vaz de Melo - DCC

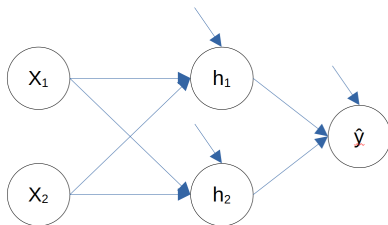
UFMG

Março de 2026

Backpropagation

- Sabemos construir a arquitetura de uma rede neural capaz de aprender funções (Aproximação Universal e MLP)
- Conhecemos um algoritmo capaz de encontrar mínimos de funções usando as informações do gradiente (Gradiente descendente)
- Precisamos de uma forma de calcular os gradientes

Backpropagation



$$h_1 = \varphi^{(1)}(w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + b_1^{(1)})$$

$$h_2 = \varphi^{(1)}(w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)})$$

$$\hat{y} = \varphi^{(2)}(w_{11}^{(2)}h_1 + w_{21}^{(2)}h_2 + b_1^{(2)})$$

Backpropagation

- Se queremos minimizar a função de perda BCE (Binary Cross Entropy):

$$L = -\frac{1}{n} \sum_{i=1}^n (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$$

- Precisamos encontrar os gradientes:

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}}, \frac{\partial L}{\partial \mathbf{b}^{(1)}}, \frac{\partial L}{\partial \mathbf{W}^{(2)}}, \frac{\partial L}{\partial \mathbf{b}^{(2)}}$$

Backpropagation

- Para modelos lineares, calculamos facilmente os gradientes
- Para redes neurais, com funções de ativação não lineares e múltiplas camadas, o desafio aumenta
- Como comentamos, fica fácil cometer erros e pouco prático/inviável
- O algoritmo de Backpropagation é uma solução para este problema

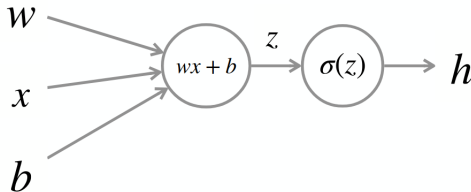
Backpropagation

- Intuitivamente, se queremos caminhar com os pesos na direção que minimiza a função de perda, precisamos saber como a função de perda se modifica ao modificarmos os pesos.
- Precisamos então, expressar a função de perda como uma função dos valores dos pesos.
- Para neurônios na camada de saída, essa avaliação é bem direta, mas o que acontece quando queremos avaliar o impacto dos pesos da primeira camada oculta?
- $\hat{y}$ é uma função composta do tipo $f(X, \Theta)$, em que Θ representa o conjunto de pesos e vieses.

Backpropagation

- Estruturar a composição de funções sobre um grafo computacional nos ajuda a não perder a direção
- Cada nó do grafo representa uma função e cada aresta representa um argumento de uma função
- Exemplo de um grafo para uma regressão logística:

$$h(w, x, b) = \sigma(wx + b)$$

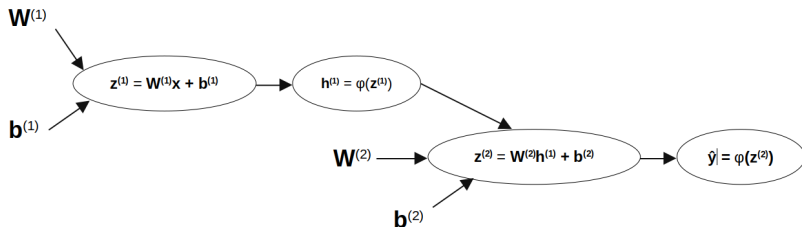


Backpropagation

- O algoritmo de Backpropagation combina a estrutura do grafo com a regra da cadeia para calcular, o gradiente de uma função com respeito a suas entradas
- No nosso caso de interesse, queremos calcular o gradiente de uma função de perda com relação aos pesos e vieses.
- O treinamento de uma rede neural, consiste então de duas etapas:
 - 1 **forward pass**: computamos a saída da rede e armazenamos os resultados parciais em cada nó do grafo
 - 2 **backward pass**: calculamos o gradiente da função de perda com respeito aos pesos e vieses

Backpropagation

Podemos reescrever nossa rede no formato de um grafo:



- Para a função de perda, introduziríamos mais um último nó
- Os vetores de x e y estão sendo deixados de fora do grafo, pois não são valores a serem aprendidos

Backpropagation

- Regra da cadeia:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \frac{dg}{dx}$$

- Regra da cadeia multivariada:

$$\frac{d}{dt} f(x(t), y(t)) = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}$$

Backpropagation

- Vamos compor o gradiente a partir da saída de trás pra frente
- Se $\hat{y} = f(x, \Theta)$, então $L = L(\Theta)$

$$\frac{\partial L}{\partial \Theta} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \Theta}$$

Pensando em um problema de classificação binária com função de perda BCE, $\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$

Precisamos então calcular $\frac{\partial \hat{y}}{\partial \Theta}$, mas sabemos que $\hat{y} = \sigma(\mathbf{z}^{(2)})$, o que leva a:

$$\frac{\partial \hat{y}}{\partial \Theta} = \frac{\partial \hat{y}}{\partial \mathbf{z}^{(2)}} \frac{\partial \mathbf{z}^{(2)}}{\partial \Theta}$$

Backpropagation

Para nossa sorte, derivando a sigmoide logística, obtemos:

$$\frac{\partial \hat{y}}{\partial \mathbf{z}^{(2)}} = \hat{y}(1 - \hat{y})$$

Como esperado, continuamos nosso caminho, lembrando que $\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}$:

$$\frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{W}^{(2)}} = \mathbf{h}^{(1)}, \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{b}^{(2)}} = 1, \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{h}^{(1)}} = \mathbf{W}^{(2)}$$

Para $\mathbf{W}^{(2)}$ e $\mathbf{b}^{(2)}$, já somos capazes de escrever os gradientes da função de perda:

$$\frac{\partial L}{\partial \mathbf{W}^{(2)}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \mathbf{z}^{(2)}} \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{W}^{(2)}} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right) \hat{y}(1-\hat{y})\mathbf{h}^{(1)} = (\hat{y}-y)\mathbf{h}^{(1)}$$

$$\frac{\partial L}{\partial \mathbf{b}^{(2)}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \mathbf{z}^{(2)}} \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{b}^{(2)}} = \hat{y} - y$$

Backpropagation

Precisamos continuar a derivação para $\mathbf{h}^{(1)} = \varphi(\mathbf{z}^{(1)})$:

$$\frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{z}^{(1)}} = \frac{\partial \varphi(\mathbf{z}^{(1)})}{\partial \mathbf{z}^{(1)}}$$

Finalmente:

$$\frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(1)}} = \mathbf{x}, \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{b}^{(1)}} = 1$$

E com isso:

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \mathbf{z}^{(2)}} \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{h}^{(1)}} \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{z}^{(1)}} \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(1)}} = \mathbf{x} \frac{\partial \varphi(\mathbf{z}^{(1)})}{\partial \mathbf{z}^{(1)}} \odot \mathbf{W}^{(2)}(\hat{y} - y)$$

$$\frac{\partial L}{\partial \mathbf{b}^{(1)}} = \frac{\partial \varphi(\mathbf{z}^{(1)})}{\partial \mathbf{z}^{(1)}} \odot \mathbf{W}^{(2)}(\hat{y} - y)$$